



Control de un Robot Balancín (Péndulo Invertido) Mediante Retroalimentación de Estados

Lucas Pin^{a,1}, Matteo Martinez^{a,1}, Franco de Vargas^{a,1}, Dr. Ing. Leonardo Comparatore^{b,1}, MSc. Ing. Christian Medina^{b,1}, MSc. Ing. Paola Maidana^{b,1}

^aEstudiante de Ingeniería Mecatrónica, Facultad de Ingeniería, Universidad Nacional de Asunción, CITEC-Luque, Paraguay

^bOrientador, Facultad de Ingeniería, Universidad Nacional de Asunción, CITEC-Luque, Paraguay

PALABRAS CLAVE

Control Optimo
LQR
Servo Sistema
Péndulo Invertido

RESUMEN

Se desarrolla el modelado y control de un robot balancín, considerado como un péndulo invertido sobre ruedas. Se parte de la obtención del modelo dinámico mediante la formulación de Lagrange, representando el sistema como cuerpos rígidos acoplados. Luego, se diseña un controlador óptimo LQR basado en realimentación de estados, con el objetivo de estabilizar la posición vertical del sistema. Finalmente, se valida el desempeño del controlador en simulación, demostrando su efectividad ante distintas condiciones iniciales.

1. Introducción

“¡Contempla, caminante del filo, la danza de un cuerpo erguido contra la tiranía de la gravedad!”.

El control de sistemas inestables ha sido un tema central en la teoría de control moderno, destacando el péndulo invertido como uno de los problemas más representativos por su naturaleza altamente no lineal y su inestabilidad inherente. En este contexto, el robot balancín, también conocido como self-balancing robot, ofrece una excelente plataforma para estudiar técnicas de control avanzadas aplicadas a un sistema real.

El objetivo principal de este trabajo es diseñar e implementar un controlador que permita mantener el equilibrio de un robot de dos ruedas con estructura vertical, utilizando retroalimentación de estados. Para ello, se parte de un modelado físico detallado de la planta, representando las distintas masas y componentes mecánicos como cuerpos rígidos acoplados. A partir de este modelo, se derivan las ecuaciones dinámicas mediante el método de Lagrange-Euler, identificando las principales variables de estado que influyen en el comportamiento del sistema.

Una vez modelado el sistema, se procede al diseño de un controlador óptimo mediante la técnica LQR (Linear Quadratic Regulator), que busca minimizar una función de costo cuadrática asociada al desempeño del sistema.

La implementación física se realizó sobre plataforma Arduino, incorporando sensores como giroscopio, acelerómetro y tacómetros para estimar las variables de estado. El control se aplicó con un driver L298N usando señal PWM.

Finalmente, se validan los resultados obtenidos tanto en simulaciones como en la implementación física del prototipo, evaluando la efectividad del control ante distintas condiciones de operación.

2. Descripción de la Planta y Modelado Matemático

2.1. Descripción de la Planta

El fenómeno físico del robot balancín se aproxima como un modelo de cuerpos rígidos acoplados, el cual consisten en dos cilindros huecos que pueden girar completamente sobre su eje (representando las ruedas), y tres prismas de masa m_1 , m_2 y m_3 (figura 1), donde la distancia perpendicular fija desde el centro de masa de estos prismas respecto al eje donde giran las ruedas (siendo este la referencia $y = 0$) esta dado por h_1 , h_2 y h_3 , como se observa en la figura 2.

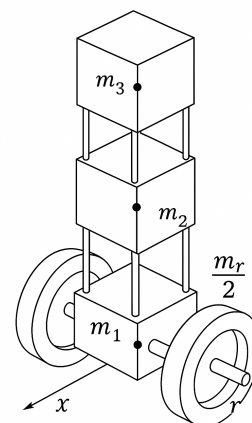


Fig. 1. Esquema Tridimensional de la Planta.

Las consideraciones tomadas para obtener sus ecuaciones dinámicas son:

- Los cilindros ruedan sin deslizar y además ambos tienen el mismo desplazamiento lineal en todo momento.

- El eje de los cilindros y los prismas están acoplados rígidamente entre si.
- La distribución de masas esta simétricamente distribuido tal que el centro de masas de los cilindros queda en un plano normal al eje de las ruedas que pasa por el punto medio del mismo.

Estas suposiciones permiten limitar la dinámica del fenómeno a un movimiento bidimensional.

2.2. Modelado Matemático por las ecuaciones de Lagrange-Euler

Para obtener las ecuaciones de su dinámica de movimiento se emplea la formulación de Lagrange-Euler. Las coordenadas generalizadas para este enfoque fueron tomadas como $q_1 = x$ y $q_2 = \theta$, donde x es la distancia horizontal de del centro del eje de las ruedas y θ es el angulo del péndulo medido desde la vertical, como se observa en la figura 2.

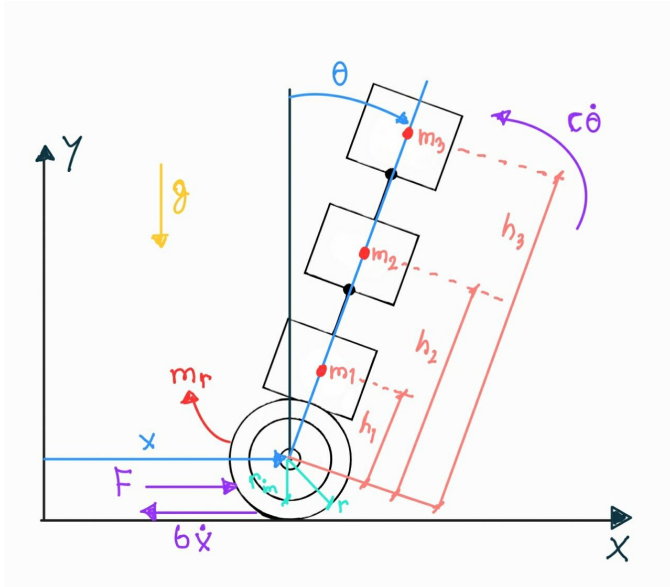


Fig. 2. Esquema Simplificado en el Plano.

Obtención de la Expresión de la Energía Cinética:

⇒ Energía Cinética de las ruedas (ruedas en rotación sin deslizamiento):

- Masa de Cada Rueda: $m_r/2$.
- Energía traslacional: $\frac{1}{2}(m_r/2)\dot{x}^2$.
- Energía rotacional (eje propio): $\frac{1}{2}I_r(\dot{x}/r)^2$, con $I_r = \frac{1}{2}(m_r/2)(r + r_{int})^2$.

Entonces la energía cinética de las ruedas está dada por

$$T_{ruedas} = 2 \left[\frac{1}{2} \cdot \frac{m_r}{2} \dot{x}^2 + \frac{1}{2} \cdot \frac{m_r}{2} r^2 \cdot \left(\frac{\dot{x}}{r} \right)^2 \right] = \frac{3}{4} m_r \dot{x}^2$$

⇒ Energía de la estructura de prismas (plataformas)

Para cada prisma (plataforma) $i \in \{1, 2, 3\}$:

- Masa: m_i .
- Distancia del CM al centro del eje de las ruedas: h_i .
- Posición del CM: $x_i = x + h_i \sin(\theta)$, $y_i = h_i \cos(\theta)$.

- Velocidad total del CM:

$$v_i = \left(\frac{dx_i}{dt} \right)^2 + \left(\frac{dy_i}{dt} \right)^2 = (\dot{x} + h_i \cos(\theta) \dot{\theta})^2 + (-h_i \sin(\theta) \dot{\theta})^2$$

Desarrollando la expresión:

$$v_i = \dot{x}^2 + 2h_i \dot{x} \dot{\theta} \cos(\theta) + h_i^2 \dot{\theta}^2$$

Entonces, energía cinética total de los prismas es:

$$T_{prismas} = \sum_{i=1}^3 \frac{1}{2} m_i v_i^2$$

$$T_{prismas} = \frac{1}{2} \left(\dot{x}^2 \sum_{i=1}^3 m_i + 2\dot{x} \dot{\theta} \cos(\theta) \sum_{i=1}^3 m_i h_i + \dot{\theta}^2 \sum_{i=1}^3 m_i h_i^2 \right)$$

Donde:

- $m_T = \sum m_i$.
- $M_h = \sum m_i h_i$.
- $I_h = \sum m_i h_i^2$.

⇒ Energía Cinética Total del Sistema:

$$T = T_{ruedas} + T_{prismas}$$

$$T = \frac{1}{2} \left(m_T + \frac{3}{2} m_r \right) \dot{x}^2 + M_h \dot{x} \dot{\theta} \cos(\theta) + \frac{1}{2} I_h \dot{\theta}^2$$

Obtención de la Expresión de la Energía Potencial:

Para el sistema considerado, la energía potencial está dada solo por la energía potencial gravitacional de las masas, esto es:

$$V = \sum_{i=1}^3 m_i h_i g \cdot \cos(\theta) = C_1 \cos(\theta)$$

Donde g es la aceleración debido a la gravedad y $C_1 = \sum m_i h_i g$.

Obtención del Lagrangiano:

El Lagrangiano está dado por:

$$\mathcal{L} = T - V$$

Entonces:

$$\mathcal{L} = \frac{1}{2} M \dot{x}^2 + M_h \dot{x} \dot{\theta} \cos(\theta) + \frac{1}{2} I_h \dot{\theta}^2 - C_1 \cos(\theta)$$

Donde $M = (m_T + \frac{3}{2} m_r)$

Ecuaciones de Lagrange-Euler:

⇒ Para $q_1 = x$

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{x}} \right) - \frac{\partial \mathcal{L}}{\partial x} = -b\dot{x} + F$$

Donde b es fricción lineal y F la fuerza neta aplicada en x .

⇒ Para $q_2 = \theta$

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{\theta}} \right) - \frac{\partial \mathcal{L}}{\partial \theta} = -c\dot{\theta}$$

Donde c es la fricción angular.

Ecuaciones de Movimiento del Sistema:

$$\begin{aligned} (1) \quad M\ddot{x} + M_h \cos(\theta)\ddot{\theta} &= F - b\dot{x} + M_h \sin(\theta)\dot{\theta}^2 \\ (2) \quad I_h\ddot{\theta} + M_h \cos(\theta)\ddot{x} &= C_1 \sin(\theta) - c\dot{\theta} \end{aligned}$$

Linealización del Sistema:

Para poder llevar la expresión de la planta al espacio de estados, se requiere de una linealización previa. Considerando la linealización al rededor del punto de equilibrio $\theta = 0$, se tiene:

- $\cos(\theta) \approx 1$.
- $\sin(\theta) \approx \theta$.
- $\dot{\theta}^2 \approx 0$.

Entonces, el sistema linealizado queda como:

$$\begin{aligned} (1') \quad M\ddot{x} + M_h\ddot{\theta} &= F - b\dot{x} \\ (2') \quad I_h\ddot{\theta} + M_h\ddot{x} &= C_1\theta - c\dot{\theta} \end{aligned}$$

Tomando los estados como:

$$X = \begin{bmatrix} x_1 & x_2 & x_3 & x_4 \end{bmatrix}^T = \begin{bmatrix} x & \dot{x} & \theta & \dot{\theta} \end{bmatrix}^T$$

Es necesario conseguir las variables $\ddot{x}$ y $\ddot{\theta}$ de (1') y (2') en función de las variables de estados x_1 , x_2 , x_3 y x_4 . Esto es posible resolviendo la ecuación matricial dada por:

$$\begin{bmatrix} M & M_h \\ M_h & I_h \end{bmatrix} \begin{bmatrix} \ddot{x} \\ \ddot{\theta} \end{bmatrix} = \begin{bmatrix} F - c\dot{x} \\ C_1\theta - c\dot{\theta} \end{bmatrix}$$

Dando como resultado:

$$\begin{aligned} \ddot{x} &= -\frac{I_h b}{\Delta} \dot{x} + \frac{M_h C_1}{\Delta} \theta - \frac{M_h c}{\Delta} \dot{\theta} + \frac{I_h}{\Delta} u \\ \ddot{\theta} &= \frac{M_h b}{\Delta} \dot{x} + \frac{M C_1}{\Delta} \theta - \frac{M c}{\Delta} \dot{\theta} - \frac{M_h}{\Delta} u \end{aligned}$$

Donde:

- $\Delta = M I_h - M_h^2$ y $F = u$ el esfuerzo de control.

Forma en el Espacio de Estados:

$$\dot{X} = A X + B u$$

Donde las matrices A y B están dadas por:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -\frac{I_h b}{\Delta} & \frac{M_h C_1}{\Delta} & -\frac{M_h c}{\Delta} \\ 0 & 0 & 0 & 1 \\ 0 & \frac{M_h b}{\Delta} & \frac{M C_1}{\Delta} & -\frac{M c}{\Delta} \end{bmatrix}$$

$$B = \begin{bmatrix} 0 \\ \frac{I_h}{\Delta} \\ 0 \\ -\frac{M_h}{\Delta} \end{bmatrix}$$

3. Diseño del Controlador

3.1. Definición del Espacio de Estados Continuo

```

1 %% 1. MODELADO DE LA PLANTA
2 % Masas de los componentes en kg
3 m_bateria = 0.125; % Masa de la bateria
4 m_arduino = 0.025; % Masa del Arduino Uno
5 m_driver = 0.025; % Masa del driver L298n
6 m MPU = 0.002; % Masa del sensor MPU
7 m_tacometro = 0.04; % Masa de cada tacometro + su disco
8 m_rueda = 0.029; % Masa de cada rueda
9 m_motor = 0.034; % Masa de cada motor
10
11 g = 9.8; % Aceleracion de la gravedad [m/s^2]
12 b = 0.1; % Friccion Lineal
13 c = 0; % Friccion Angular (Despreciado)
14 % --- PARAMETROS (1)---
15 % Datos de la rueda
16 mr = 2*m_rueda;
17 r = 3.4/100;
18 r_int = 2.5/100;
19 Ir = (1/2)*m_rueda*(r^2+r_int^2); %Momento de Inercia
20
21 % Datos del prima 1 (Bateria + Motores)
22 m1 = m_bateria + 2*m_motor + 2*m_tacometro;
23 h1 = 0.1/100;
24
25 % Datos del prima 2 (Arduino + Drivers)
26 m2 = m_arduino + m_driver;
27 h2 = 5.2/100;
28
29 % Datos del prima 3 (Sensor MPU)
30 m3 = m MPU;
31 h3 = 10/100;
32
33 % --- PARAMETROS SIMPLIFICADOS---
34 mT = m1 + m2 + m3;
35 Mh = m1*h1 + m2*h2 + m3*h3;
36 Ih = m1*h1^2 + m2*h2^2 + m3*h3^2;
37 M = mT + (3/2)*mr;
38 C1 = g*Mh;
39 delta = M*Ih-Mh^2;
40 %% 2. DEFINICION DEL MODELO EN ESPACIO DE ESTADOS
41 a11 = 0;
42 a12 = 1;
43 a13 = 0;
44 a14 = 0;
45
46 a21 = 0;
47 a22 = -(Ih*b)/delta;
48 a23 = (Mh*C1)/delta;
49 a24 = -(Mh*c)/delta;
50
51 a31 = 0;
52 a32 = 0;
53 a33 = 0;
54 a34 = 1;
55
56 a41 = 0;
57 a42 = (Mh*b)/delta;
58 a43 = (M*C1)/delta;
59 a44 = -(M*c)/delta;
60
61 b1 = 0;
62 b2 = (Ih)/delta;
63 b3 = 0;
64 b4 = -(Mh)/delta;
65
66 A = [a11 a12 a13 a14;
67      a21 a22 a23 a24;
68      a31 a32 a33 a34;
69      a41 a42 a43 a44]; % matriz A del sistema
70 B = [b1 b2 b3 b4]'; % matriz B del sistema
71 C = [1 0 0 0]; % tomando como entrada x
72 D = 0;

```

Teniendo como resultado las matrices:

$$A = \begin{bmatrix} 0 & 1,0000 & 0 & 0 \\ 0 & -0,2847 & 1,6946 & 0 \\ 0 & 0 & 0 & 1,0000 \\ 0 & 5,6270 & 227,1964 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 \\ 2,8469 \\ 0 \\ -56,2702 \end{bmatrix}$$

Y tomando la el estado $x_1 = x$ (esto es debido que esto es la unica combinacion que hace posible que el sistema sea observable), la distancia lineal, la matriz C queda como:

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix} \quad y \quad D = 0$$

3.2. Metodo de Discretizacion

Para la discretización se tomo un tiempo de muestro de $T_s = 10 \text{ ms}$, adecuado para su implementación en placas de desarrollo como Arduino. El método de discretización usado fue el **ZOH (Zero-Order Hold)** esto es debido a que refleja fielmente el comportamiento de cómo se mantienen constantes las señales de control entre actualizaciones en sistemas digitales reales.

```
1 %% 2. DISCRETIZACION
2 Ts = 0.01; % Tiempo de muestreo 10ms
3
4 sys_C = ss(A,B,C,D); % Planta Continua
5 sys_D = c2d(sys_C, Ts, 'zoh'); % Planta Discreta
6
7 % Matrices en el Espacio de Estado Discreto
8 G = sys_D.A;
9 H = sys_D.B;
10 C = sys_D.C;
11 D = sys_D.D;
```

Teniendo como resultado las matrices en el espacio de estados discreto:

$$G = \begin{bmatrix} 1,0000 & 0,0100 & 0,0001 & 0,0000 \\ 0 & 0,9972 & 0,0170 & 0,0001 \\ 0 & 0,0003 & 1,0114 & 0,0100 \\ 0 & 0,0564 & 2,2811 & 1,0114 \end{bmatrix}$$

$$H = \begin{bmatrix} 0,0001 \\ 0,0284 \\ -0,0028 \\ -0,5640 \end{bmatrix}$$

Teniendo el sistema discreto se puede comprobar que **cumple con ser completamente controlable** y es observable solo cuando se toma como entrada de retroalimentación la posición lineal x (el cual es el caso estudiado en este trabajo).

3.3. Diseño y Calculo del Controlador

El sistema dado nos permite diseñar un servo sistema observable. Debido que la retroalimentación debemos hacerlo con la posición lineal x para poder observar las demás variables y esta no podemos medirla directamente (se requiere de una integración numérica de las lecturas obtenidas por los taco-metros de las ruedas) y demás la calidad de estimación del observador depende de la fidelidad del modelo físico empleado **se opto por usar un servo sistema sin observador**.

El calculo de para la matriz K de retroalimentación de estados fue realizado usando el método **LQR (linear Quadratic Regulator)**

```
1 %% 4. DISEÑO DE LA REALIMENTACION DE ESTADOS
2 % Penalizacin alta para x y theta
3 Q = diag([200, 10, 200, 20]);
4 R = 1; % Penalizacin para el esfuerzo de control
5
6 K = dlqr(G,H,Q,R);
```

Teniendo como resultado:

$$K = \begin{bmatrix} -4,7152 & -3,3278 & -17,0580 & -1,8229 \end{bmatrix}$$

3.4. Ecuación en Diferencias que Define al Sistema Retroalimentado

La dinámica de los estados se actualiza según la siguiente expresión:

$$X[k+1] = GX[k] + H(-KX[k])$$

Siendo el esfuerzo de control:

$$u[k] = -KX[k]$$

3.5. Resultados de la Simulación

A continuación se muestran los resultados de la respuesta del sistema sin retroalimentación de estados y con retroalimentación de estados. Para una inclinación de $\theta = 10^\circ \approx 0,1745 \text{ rad}$. Siendo entonces el estado inicial:

$$X[0] = \begin{bmatrix} x_1[0] \\ x_2[0] \\ x_3[0] \\ x_4[0] \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0,1745 \\ 0 \end{bmatrix}$$

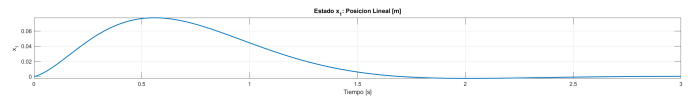


Fig. 3: Evolución de posición lineal [m] a una inclinación inicial de 10° con retroalimentación de estados

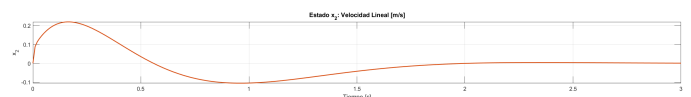


Fig. 4: Evolución de velocidad lineal [m/s] a una inclinación inicial de 10° con retroalimentación de estados



Fig. 5: Evolución del ángulo del péndulo [rad] a una inclinación inicial de 10° con retroalimentación de estados



Fig. 6: Evolución de la velocidad angular del péndulo [rad/s] a una inclinación inicial de 10° con retroalimentación de estados

4. Implementación del Controlador en el Robot Balancín

4.1. Sensores y Drivers Usados

Para la estimación de las variables de estado del sistema se utilizaron tres sensores: un giroscopio, un acelerómetro y dos tacómetros (uno por cada rueda). La velocidad lineal del sistema se obtuvo a partir del promedio de las velocidades angulares de los tacómetros, multiplicado por el radio de las ruedas. La posición lineal fue estimada mediante integración numérica de dicha velocidad.

El ángulo del péndulo se obtuvo principalmente del giroscopio. Sin embargo, para reducir el error y el ruido en la medición, se aplicó un filtro de Kalman 1D combinando los datos del acelerómetro, seguido de un filtro complementario para suavizar la señal. La velocidad angular del péndulo fue estimada a partir de estas señales mediante integración y filtrado adicional.

El control de los motores se implementó mediante un driver L298N, donde la entrada PWM corresponde al esfuerzo de control u generado por el controlador, previamente escalado. Esto permite controlar tanto la dirección como la magnitud del movimiento del sistema.

4.2. Implementación del Controlador en Arduino

```

1 void loop() {
2   /// DEFINICION DE LA K DE RETROALIMENTACION ///
3   float k1 = -4.7152;
4   float k2 = -3.3278;
5   float k3 = -17.0580;
6   float k4 = -1.8229;
7
8   /// OBTENCION DE LAS VARIABLES DE ESTADOS ///
9   gyro.update(); // Aactualiza la lectura del Giroscopio
10  // Obtiene el angulo theta
11  float Pitch_angle = gyro.get_PitchAngle();
12
13  // Obtiene la velocidad angular del pendulo:
14  float Pitch_rate = gyro.get_PitchAngle_Rate();
15  // Filtro Complementario para el angulo
16  Pitch_rate = (0.6* Pitch_rate+ (1-0.6)*prev_Pitch)-3;
17  prev_Pitch= Pitch_rate;
18
19  tacoA.update(); // Acutiliza el tacometro de la rueda A
20  tacoB.update(); // Acutiliza el tacometro de la rueda B
21  // Toma la velocidad como el promedio
22  float w_ruedas = (tacoA.get_angular_rate()+
23                    tacoB.get_angular_rate())/2;
24  // Velocidad lineal de las ruedas
25  float v_lineal = w_ruedas*(3.4/100);
26
27  integrador.setVelocity(sing*v_lineal);
28  integrador.update();
29  // La posicion se consigue integrando la velocidad
30  float x = integrador.getPosition();
31
32  /// LEY DE CONTROL ///
33  // Muestreo de 10 ms
34  if (millis() - t_ant >= 10){
35
36    t_ant = millis();
37
38    // Calculo del esfuerzo de control
39    u = -(x*k1+v_lineal*k2+Pitch_angle*k3+Pitch_rate*k4);
40
41  }
42
43  // Control de los motores con u
44  Control_Motors(u);
45 }

```

4.3. Resultados de la Implementación

```

//////////
ESTADOS
x:-0.23
v_lineal:0.60
tita:-11.80
tita_rate:-24.79
Esfuerzo de Control
u:-247.58
//////////
ESTADOS
x:-0.54
v_lineal:0.60
tita:-16.48
tita_rate:-9.10
Esfuerzo de Control
u:-299.74
//////////
ESTADOS
x:-0.80
v_lineal:0.80
tita:-1.74
tita_rate:19.39
Esfuerzo de Control
u:7.23
//////////
ESTADOS
x:-1.14
v_lineal:0.80
tita:-0.85
tita_rate:7.25
Esfuerzo de Control
u:-4.04

```

Fig. 7. Resultados de la Implementación en Arduino (θ medido en grados)

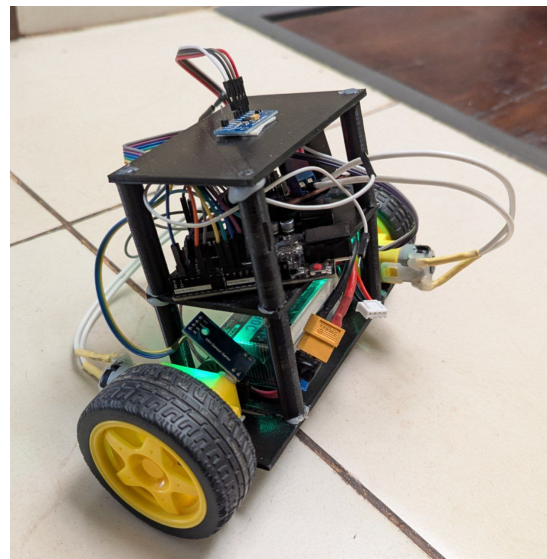


Fig. 8. Robot Balancín Prototipo "Paco"

5. Conclusiones

Este trabajo logro dejar en evidencia la eficacia de los métodos modernos de control, tal como es la retroalimentación de estados y tiene gran importancia en sistemas del tipo MISO que ademas sean inherentemente inestables, como es el caso del Robot Balancín, estos mismos sistemas tienen un control pobre o nulo con técnicas de control clásicas como es el PID la cual es un control pensado exclusivamente para sistemas del tipo SISO.

6. Referencias

- [1] Katsuhiko Ogata. *Discrete-Time Control Systems*. 2nd ed., Prentice Hall, Upper Saddle River, New Jersey, 1995.
- [2] Ramírez-González, L., García-Martínez, J., Siordia-Vásquez, X. y García-Ramos, R. (2017). *Euler-Lagrange en el modelado y control de un péndulo invertido sobre un carro*. Revista de Ingeniería Eléctrica, 1(3), 1-8. ECFORFAN.